Map Coloring Problem Using CSP

Aim

To implement the **Map Coloring Problem** using a **Constraint Satisfaction Problem (CSP)** in Python.

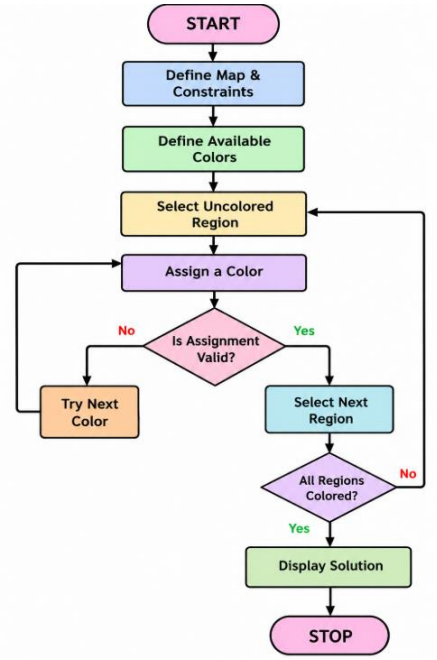
Objectives

- To understand the concept of CSP.
- To assign different colors to connected regions.
- To ensure adjacent regions have different colors.
- To implement backtracking for finding a valid solution.

Algorithm

1. Start.
2. Define the map regions and their neighboring regions.
3. Define the available colors.
4. Select an uncolored region.
5. Assign a color to the region.
6. Check whether the assignment satisfies all constraints.
7. If valid, move to the next region.
8. If invalid, try another color.
9. Backtrack when no valid color is available.
10. Display the final coloring.
11. Stop.

Flowchart



Program

```
colors = ["Red", "Green", "Blue"]

graph = {
    "A": ["B", "C"],
    "B": ["A", "C", "D"],
    "C": ["A", "B", "D"],
    "D": ["B", "C"]
}

assignment = {}

def is_safe(region, color):
    for neighbor in graph[region]:
        if neighbor in assignment and assignment[neighbor] == color:
            return False
    return True

def solve():
    if len(assignment) == len(graph):
        return True

    region = next(r for r in graph if r not in assignment)

    for color in colors:
        if is_safe(region, color):
            assignment[region] = color

            if solve():
                return True

            del assignment[region]

    return False

if solve():
    print("Map Coloring Solution:")
    for region, color in assignment.items():
        print(region, ":", color)
else:
    print("No solution exists")
```

output

```
Map Coloring Solution:
A : Red
B : Green
C : Blue
D : Red
```

Result

The **Map Coloring Problem** was successfully implemented using the **Constraint Satisfaction Problem (CSP)** approach with backtracking, and a valid coloring was obtained where adjacent regions have different colors.